

Optimising Weak Reference Processing in OpenJDK's ZGC

Master's Thesis in Computer Science

Fredrik Hammarberg

MSc Computer and Information Engineering, Uppsala University

fredrik.hammarberg00@outlook.com

May 2026

Master's Thesis Defence – Uppsala University



UPPSALA
UNIVERSITET

Agenda

- 1 Background
- 2 Motivation
- 3 Design & Implementation
- 4 Evaluation
- 5 Results
- 6 Conclusions



Background

Java and the JVM

- ▶ One of the most used programming languages in the world
- ▶ Write once, compile once – runs on *any* machine via the **JVM**
- ▶ **HotSpot JVM** is the reference implementation (Oracle / OpenJDK)



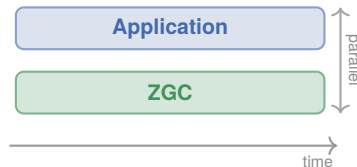
Garbage Collection

- ▶ Java has **automatic memory management** → no `free()` or `delete`
- ▶ The GC periodically finds objects the application can no longer reach and reclaims their memory

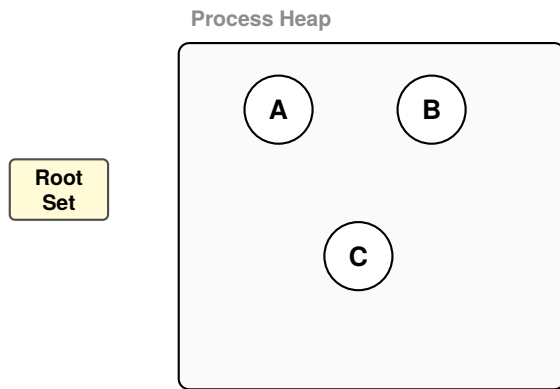
ZGC

- ▶ **Concurrent:** runs *alongside* the application → very short stop-the-world pauses
- ▶ **Relocating:** moves live objects to new compact regions. Old memory reused

ZGC runs concurrently

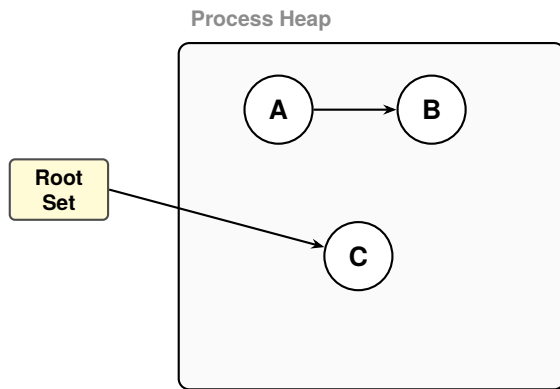


Garbage Collection



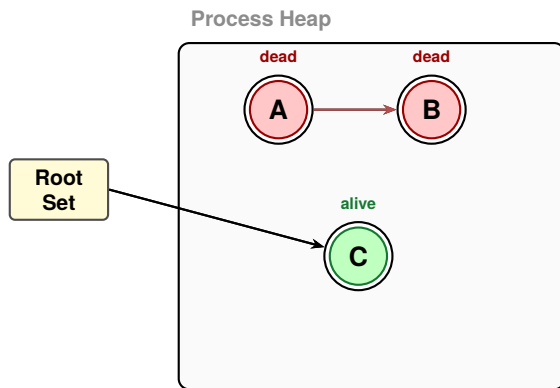
- Objects live on the **heap**, the root set is always alive

Garbage Collection



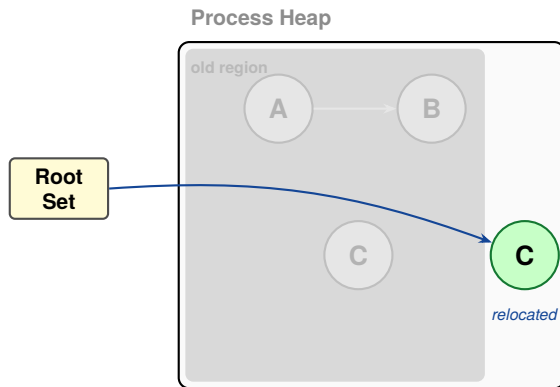
- ▶ Objects live on the **heap**, the root set is always alive
- ▶ GC **marks**: follows all references reachable from roots

Garbage Collection



- ▶ Objects live on the **heap**, the root set is always alive
- ▶ GC **marks**: follows all references reachable from roots
- ▶ **A** and **B** are unreachable → marked as **dead**. **C** is reachable → marked as **alive**

Garbage Collection



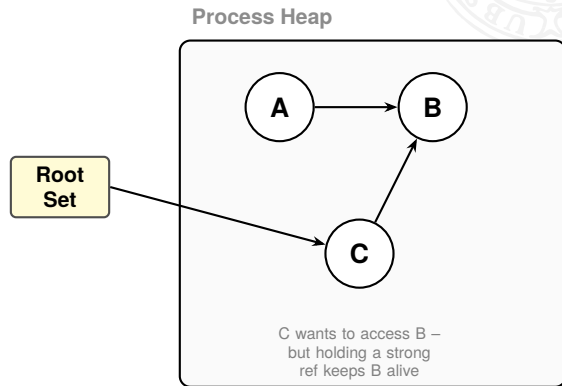
- ▶ Objects live on the **heap**, the root set is always alive
- ▶ GC **marks**: follows all references reachable from roots
- ▶ **A** and **B** are unreachable → marked as **dead**. **C** is reachable → marked as **alive**
- ▶ GC **relocates** C to a new region; old memory freed

Weak References

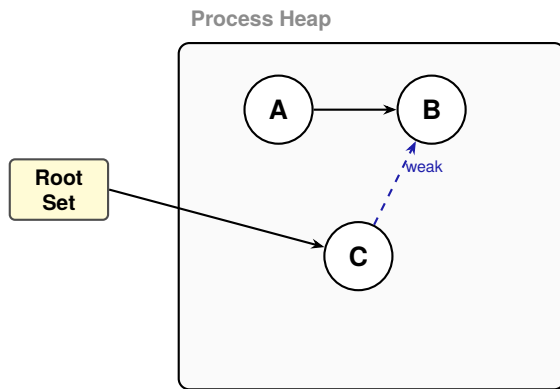
- ▶ Strong references keep objects alive. If you can reach it, it won't be collected
- ▶ But sometimes you want to **observe** an object *only while it is still alive*
- ▶ If you hold a strong ref just to check whether it's alive, you keep it alive forever, defeating the purpose

The problem

You want to hold a reference that **does not** prevent garbage collection but still lets you read the object if it hasn't been collected yet.

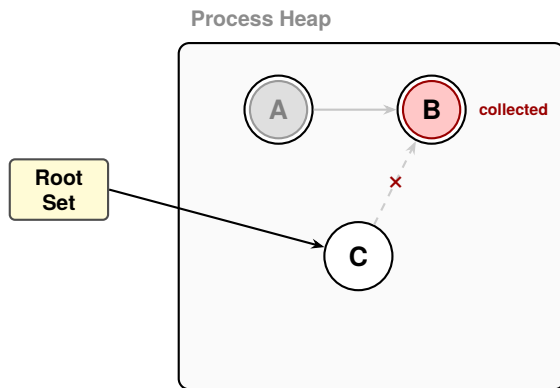


Weak References



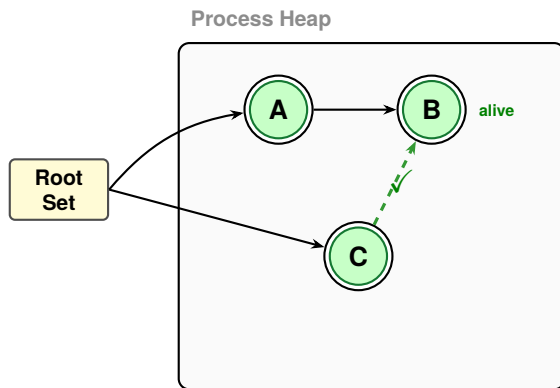
- ▶ A **weak reference** does not keep an object alive since it does not count as reachability for GC purposes

Weak References



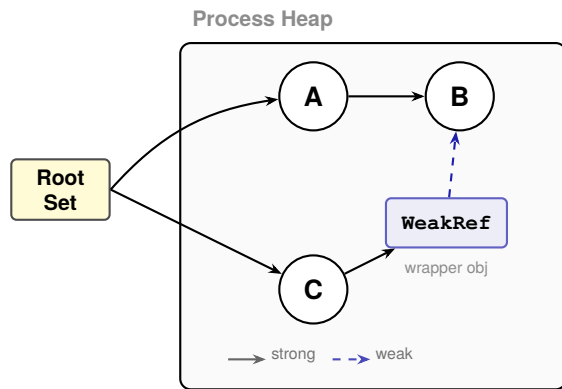
- ▶ A **weak reference** does not keep an object alive since it does not count as reachability for GC purposes
- ▶ If B has no *strong* path from roots: GC collects B; weak ref becomes `null`

Weak References



- ▶ A **weak reference** does not keep an object alive since it does not count as reachability for GC purposes
- ▶ If B has no *strong* path from roots: GC collects B; weak ref becomes `null`
- ▶ Add strong ref between root and A: B reachable through A → The weak ref from C to B is **valid**

The WeakReference Wrapper Object



- ▶ The wrapper object exists to support the `ReferenceQueue` callback
- ▶ When the GC collects B, the `WeakReference` is enqueued so the application can react

```
// With notification queue  
new WeakReference<>(obj, queue)  
  
// No queue  
new WeakReference<>(obj)
```

Reference Processing Pipeline

1. Discovery Phase

Mark phase: `WeakReferences`
appended to a *discovered list*

2. Processing Phase

Referent alive? → keep
Referent dead? → clear it

3. Enqueue Phase

`ReferenceHandler` thread
delivers each cleared ref
to its `ReferenceQueue`

- ▶ During marking: found `WeakReferences` go onto a linked list
- ▶ After marking: check liveness per ref, clear dead ones
- ▶ Cleared refs handed to the `ReferenceHandler` thread for queue delivery
- ▶ Queue-less refs travel through all three phases – silently dropped at step 3

Motivation

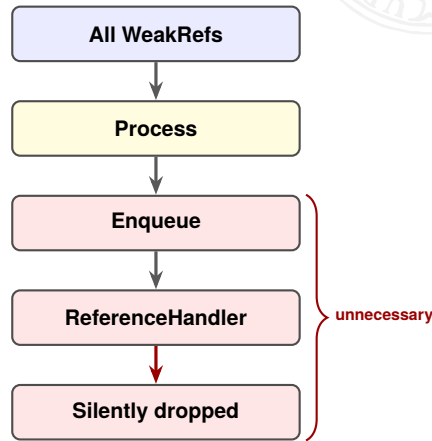
The Problem with Queue-less Weak References

- ▶ GC performance directly impacts application performance
- ▶ ZGC was designed for high performance – every unnecessary operation matters

The issue

When a `WeakReference` *without* a `ReferenceQueue` is cleared, it is still handed to the `ReferenceHandler` thread – which then silently drops it, since there is no queue.

Unnecessary work for every single queue-less weak reference.



Research Questions

RQ1 – Pipeline Optimisations

How do specific modifications to ZGC's processing of `WeakReference` objects *without* a `ReferenceQueue` affect reference-processing time, total GC cycle time, GC memory footprint, and Java heap usage?

RQ2 – Weak Fields

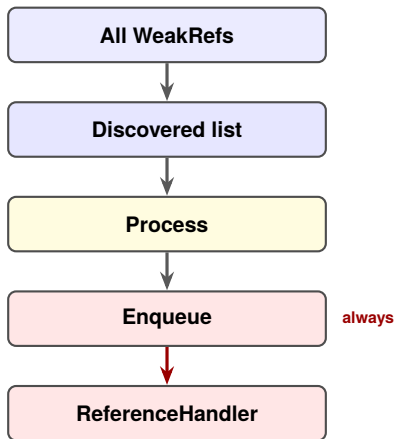
How does expressing weak semantics through annotated object fields affect reference processing time, GC cycle time, GC memory footprint, and Java heap usage compared to the optimised `WeakReference`-based approach of RQ1?

RQ1: optimise the *existing* pipeline without changing how weak refs are represented

RQ2: eliminate the `WeakReference` wrapper object entirely

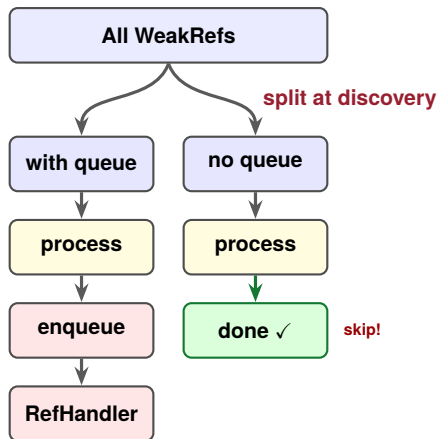
Design & Implementation

Optimisation 1: Skip Enqueue Path



- ▶ Every weak ref goes through the full pipeline
- ▶ Queue-less refs still hit the `ReferenceHandler` – which drops them
- ▶ Pure waste for weak references without a queue

Optimisation 1: Skip Enqueue Path

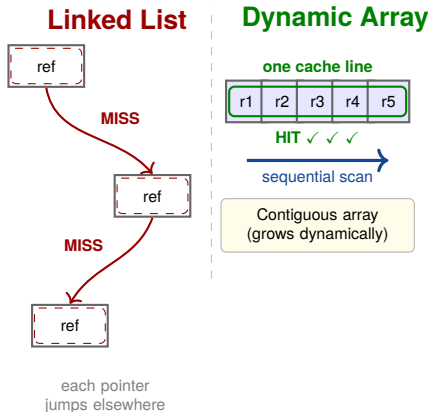


- ▶ At discovery: check the `ReferenceQueue` field
- ▶ Route queue-less refs to a *separate* discovered list
- ▶ No-queue pipeline: process (check + clear) – then **stop**
- ▶ With-queue: full existing behaviour

Why two lists instead of a branch?

A branch requires a memory load per ref to check the queue field. Two specialised loops avoid that per-reference cost.

Optimisation 2: Dynamic Array Discovered List

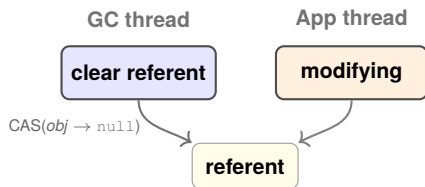


- ▶ Linked list: every next-node pointer is a **cache miss**
- ▶ Nodes scattered across memory – poor spatial locality
- ▶ Array: one cache load fills several consecutive entries
- ▶ Each append is $\mathcal{O}(1)$ amortised cost

Trade-off

Higher auxiliary GC memory for array-based variants – explored in the Results section.

Optimisation 3: Optimised Clear Path



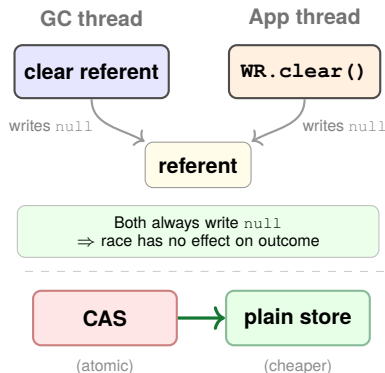
Compare-And-Swap (CAS):

```
if (*field == expected)
    *field = new_val; // atomic
```

Guarantees only one writer wins the race.

- ▶ Clearing a referent = writing `null` to the referent field
- ▶ The GC does this with a **CAS** – an atomic instruction that only writes if the current value matches an expected value
- ▶ Needed because the ZGC runs concurrently with the application – both may access the referent field at the same time without explicit synchronisation.

Optimisation 3: Optimised Clear Path



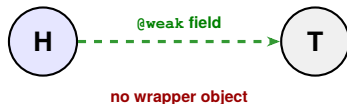
- ▶ **Key observation:** the only thing an app thread can write to a `WeakReference`'s referent field is `null` (via `WeakReference.clear()`)
- ▶ If a race occurs, both sides write `null` – the result is identical either way
- ▶ ⇒ The CAS is unnecessary; a **plain store** is safe

Research Question 2: Weak Fields (@weak)

WeakReference implementation



@weak field implementation



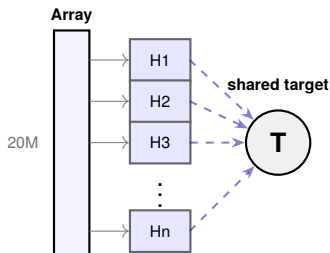
- ▶ A field annotated `@weak` is treated as a weak reference by ZGC – no wrapper object needed
- ▶ Discovered during marking, stored in a dynamic array of weak fields
- ▶ Cleared the same way – but **cannot** use the optimised clear path (a regular field can be written to with any value, not just `null`)

```
class HolderClass {  
    @weak TargetClass ref;  
}
```

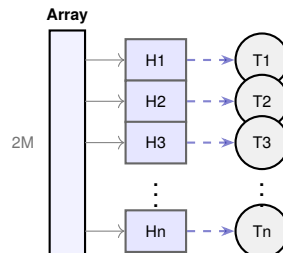
Evaluation

Benchmark Design

Single-Object Benchmark



Multi-Object Benchmark



- ▶ **Single:** 20M holders, one shared payload – all 20M refs processed in one GC cycle
- ▶ **Multi:** 2M holders, individual payloads, 5 gradual rounds – more realistic
- ▶ **Both:** no ReferenceQueue; both have a @weak field counterpart

Variants and Execution

Variant	clr	sep	dyn
none	—	—	—
dyn_only	—	—	✓
sep_only	—	✓	—
sep_dyn	—	✓	✓
clear_path_only	✓	—	—
clear_path_dyn	✓	—	✓
clear_path_sep	✓	✓	—
all	✓	✓	✓
weak_fields	—	—	—

- All $2^3 = 8$ pipeline combinations plus `weak_fields`

Execution Environment

UPPMAX supercomputer (Pelle)

AMD EPYC 9454P (Zen 4)

48 physical cores, 768 GiB RAM

4 parallel instances per run

Each: 10 JVM cores + 2 aux cores

100 GB heap `-XX:+UseZGC`

250 iterations (+1 warmup discarded)

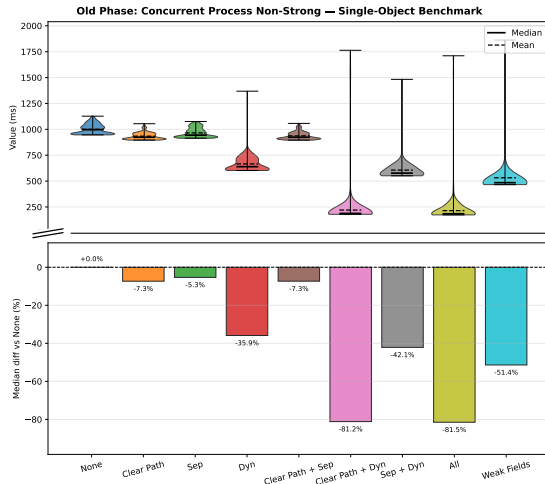
Reporting

Median over 250 iterations.

Violin + median percentage-difference histogram.

Results

Non-Strong Processing Time



Variant

Reduction

sep_only	-5%
clear_path_only	-7%
dyn_only	-36%
clear_path_dyn	-81%
all	-81%
weak_fields	-51%

Super-additive interaction

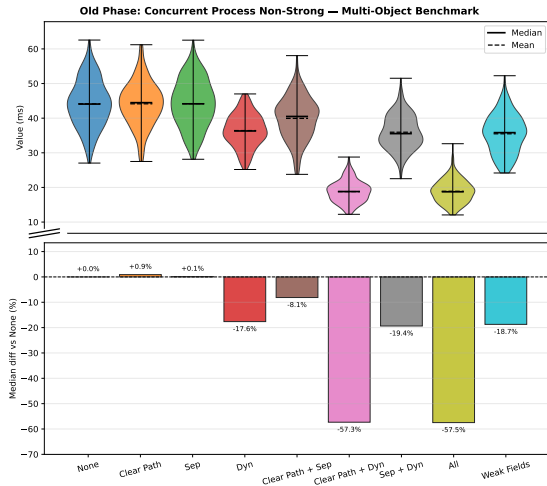
clear_path alone: -7%

dyn alone: -36%

Together: -81%

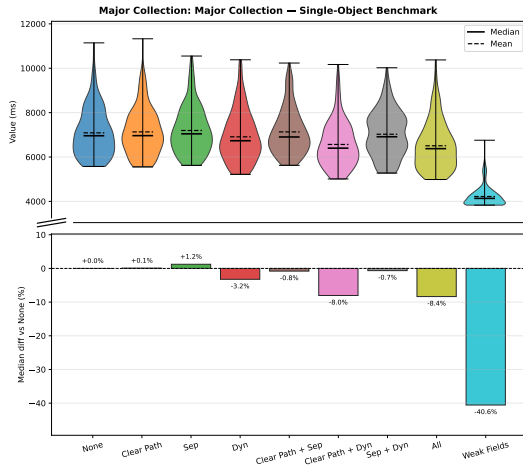
Each removes a different bottleneck.

Non-Strong Processing – Multi-Object & weak_fields

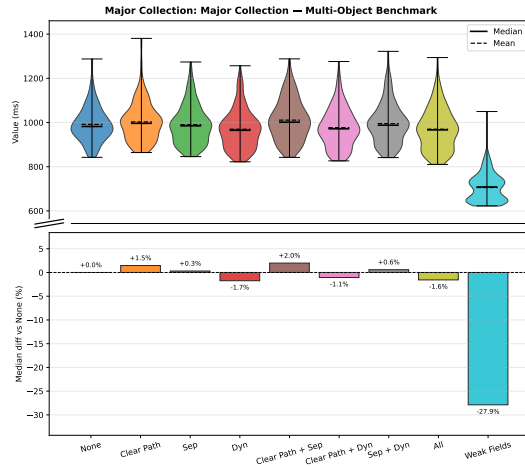


- ▶ Multi-object: same pattern – `clear_path_dyn` and `all` at -57%
- ▶ `sep_only`: 0% – the ReferenceHandler overhead is not captured by this metric
- ▶ `weak_fields`: -51% (single), -19% (multi)
- ▶ Comparable to `sep_dyn` – because `@weak` fields cannot use the optimised clear path

Major Collection Time



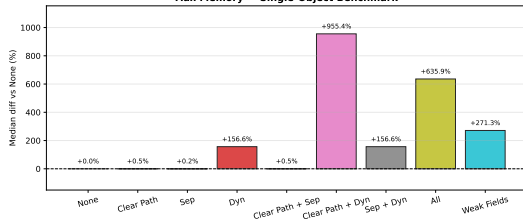
Single-object benchmark



Multi-object benchmark

Memory Usage

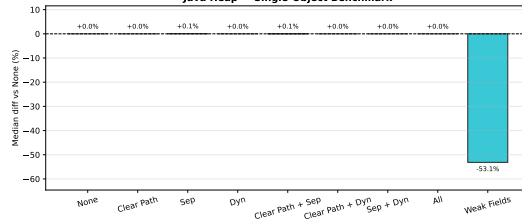
Aux Memory — Single-Object Benchmark



Auxiliary GC memory – single-object benchmark

- ▶ `clear_path_dyn`: ×10 aux memory (1268 MB vs 120 MB)
- ▶ `all`: ×7 (sep reduces bytes per entry vs dyn alone)

Java Heap — Single-Object Benchmark



Java heap – single-object benchmark

- ▶ All WeakReference variants: heap essentially identical
- ▶ `weak_fields`: -53% heap (single), -5% (multi) – no wrapper objects to allocate and promote

Conclusions

Conclusions

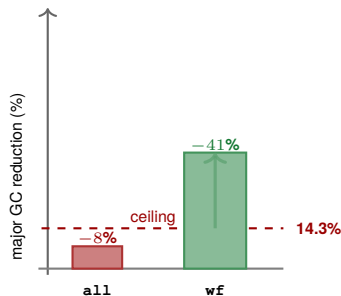
RQ1 – Pipeline Optimisations

- ▶ **81%** reduction in non-strong processing time (`clear_path_dyn / all`)
- ▶ Translates to only **8%** in total major collection time
- ▶ Hard ceiling: non-strong is 14.3% of major GC time (single) and 4.5% (multi)

RQ2 – Weak Fields

- ▶ **41%** major GC time reduction (single), **28%** (multi)
- ▶ **53%** heap reduction (single) – no wrapper objects
- ▶ Structural advantage: fewer objects means less work across *every* GC phase

Key Takeaway



Optimise the pipeline (`all`)

–81% non-strong, but only –8% total GC

Hard ceiling: non-strong is only 14% of major GC

Change the representation (`weak_fields`)

–41% total GC, –53% Java heap

Fewer objects → less work in every GC phase

The bottleneck is the *representation*

Rethink how weak semantics is represented –
not just how to process the wrappers faster.

Thank You

Questions?

fredrik.hammarberg00@outlook.com

References I

[◀ Back to start](#)

- [1] OpenJDK, “Jep 333: Zgc: A scalable low-latency garbage collector (experimental).” <https://openjdk.org/jeps/333>, 2018.
Accessed: 2026-03-12.
- [2] OpenJDK, “Jep 439: Generational zgc.” <https://openjdk.org/jeps/439>, 2023.
Accessed: 2026-03-12.
- [3] E. Österlund, “JVMLS – generational ZGC and beyond.” <https://inside.java/2023/08/31/generational-zgc-and-beyond/>, 2023.
Accessed: 2026-05-25.



References II

- [4] Oracle, “java.lang.ref (java se 25).”
<https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ref/package-summary.html>, 2025.
Accessed: 2026-03-12.
- [5] OpenJDK Bug System, “Unnecessary pending-list traversal for references without referencequeue.”
<https://bugs.openjdk.org/browse/JDK-8029205>, n.d.
Accessed: 2026-02-24.
- [6] U. Drepper, “What every programmer should know about memory,” tech. rep., Red Hat, Inc., 2007.
Accessed: 2026-04-07.



References III

- [7] R. Jones, A. Hosking, and E. Moss, *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.
- [8] H. Lieberman and C. Hewitt, “A real time garbage collector based on the lifetimes of objects,” Tech. Rep. AIM-569a, MIT Artificial Intelligence Laboratory, Oct. 1981.
Accessed: 2026-02-24.
- [9] F. Hammarberg, “Benchmark data: Optimising weak reference processing in the jvm z garbage collector.” Zenodo, 2026.
Dataset. DOI to be assigned upon publication.